

Python pour la Data Science

Guide pratique - Du débutant au Data Scientist

Syntaxe, exercices corrigés, sorties commentées

Ce qu'un Data Scientist doit savoir en Python après une formation Fullstack

PARTIE 1 - Les bases Python à maîtriser absolument

Python est le langage de référence en Data Science. Avant de toucher à NumPy, Pandas ou scikit-learn, ces bases doivent être solides. Cette partie couvre tout ce qu'un Data Scientist rencontre au quotidien.

1.1 Fonctions

Une fonction est un bloc de code réutilisable. On la déclare une fois, on l'appelle autant de fois qu'on veut. C'est la brique de base de tout programme Python.

Structure d'une fonction :

- **def** : mot-clé pour déclarer une fonction (obligatoire)
- **nom_fonction** : le nom qu'on choisit, en minuscules avec underscores
- **paramètres** : les entrées de la fonction (optionnels)
- **return** : la valeur que la fonction renvoie (sans return, elle renvoie None)
- L'indentation (4 espaces) est OBLIGATOIRE en Python

```
# Déclaration d'une fonction simple
def addition(a, b):
    resultat = a + b
    return resultat

# Appel de la fonction
print(addition(3, 4))
print(addition(10, 20))
```

Sortie :

```
7
30
```

```
# Fonction avec valeur par défaut
# Si on n'indique pas 'message', il vaut 'Bonjour' automatiquement
def saluer(nom, message='Bonjour'):
    return f'{message}, {nom}!'

print(saluer('Alice'))
print(saluer('Alice', 'Salut'))
print(saluer('Alice', 'Bonsoir'))
```

Sortie :

```
Bonjour, Alice!
Salut, Alice!
Bonsoir, Alice!
```

```
# Fonction qui retourne plusieurs valeurs (tuple)
def statistiques(liste):
    minimum = min(liste)
    maximum = max(liste)
    moyenne = sum(liste) / len(liste)
    return minimum, maximum, moyenne

mini, maxi, moy = statistiques([3, 1, 4, 1, 5, 9])
print(mini)
print(maxi)
print(round(moy, 2))
```

Sortie :

```
1
9
3.83
```

Piège classique

Une fonction sans return renvoie None (rien).

```
def ma_fonction(x):
```

```
    x * 2    # pas de return !
```

```
print(ma_fonction(5)) # affiche : None
```

Solution : toujours vérifier que return est présent quand on veut un résultat.

1.2 Types de données fondamentaux

Python a plusieurs types de données natifs. Les connaître est indispensable car chaque type a ses propres méthodes et comportements.

list [1, 2, 3]	Ordonné, modifiable. On peut ajouter, supprimer, modifier des éléments librement.
dict {'a': 1}	Paires clé-valeur. Accès instantané par clé. Très utilisé pour structurer des données.
tuple (1, 2, 3)	Ordonné, IMMuable. Une fois créé, on ne peut plus le modifier. Plus rapide que list.
set {1, 2, 3}	Valeurs uniques, non ordonné. Parfait pour dédoublonner ou tester l'appartenance.
str 'bonjour'	Chaîne de caractères. Immuable. Nombreuses méthodes : .upper(), .split(), .strip()...
int 42	Nombre entier. Opérations : +, -, *, /, //, %, ** (puissance).
float 3.14	Nombre décimal. Attention aux arrondis : 0.1 + 0.2 ≠ 0.3 exactement en Python.
bool True/False	Booléen. Résultat de comparaisons. True vaut 1, False vaut 0 en calcul.

```
# Exemples de chaque type
ma_liste = [10, 20, 30, 40]      # liste
mon_dict = {'nom': 'Alice', 'age': 28} # dictionnaire
mon_tuple = (1, 2, 3)           # tuple immuable
mon_set = {1, 2, 2, 3, 3}       # set : doublons supprimés

# Accès aux éléments
print(ma_liste[0])              # premier élément
print(ma_liste[-1])             # dernier élément
print(ma_liste[1:3])            # éléments index 1 et 2 (slice)
print(mon_dict['nom'])           # valeur par clé
print(mon_dict.get('age', 0))   # 0 si clé absente
print(mon_set)
```

Sortie :

```
10
40
[20, 30]
Alice
28
{1, 2, 3}
```

```
# Modifier une liste
ma_liste.append(50)           # ajouter en fin
ma_liste.insert(0, 5)         # insérer à l'index 0
ma_liste.remove(20)           # supprimer la valeur 20
ma_liste.pop()                # supprimer le dernier élément
print(ma_liste)

# Modifier un dictionnaire
mon_dict['ville'] = 'Paris'    # ajouter une clé
mon_dict['age'] = 29           # modifier une valeur
```

```
del mon_dict['nom']          # supprimer une clé
print(mon_dict)

# Vérifier le type d'une variable
print(type(42))             # <class 'int'>
print(type('hello'))       # <class 'str'>
print(isinstance(42, int))  # True
```

Sortie :

```
[5, 10, 30, 40]
{'age': 29, 'ville': 'Paris'}
<class 'int'>
<class 'str'>
True
```

1.3 Opérateurs et conditions

Les opérateurs sont les outils de base pour comparer des valeurs et construire des conditions.

== égalité	<code>x == 5</code> True si x vaut 5
!= différent	<code>x != 5</code> True si x ne vaut pas 5
> >= <=	comparaisons numériques classiques
and ET logique	<code>x > 0 and x < 10</code> - PAS && - les deux doivent être vrais
or OU logique	<code>x < 0 or x > 100</code> - PAS - au moins un doit être vrai
not NON logique	<code>not x == 0</code> - PAS ! - inverse la condition
in appartenance	<code>'a' in ['a','b','c']</code> True si l'élément est dans la liste
is identité	<code>x is None</code> teste si x est exactement None

```
# Structure if / elif / else
note = 14

if note >= 16:
    mention = 'Très Bien'
elif note >= 14:
    mention = 'Bien'
elif note >= 12:
    mention = 'Assez Bien'
elif note >= 10:
    mention = 'Passable'
else:
    mention = 'Échec'

print(f'Note : {note}/20 - Mention : {mention}')

# Condition sur plusieurs critères
age = 25
revenu = 3500
if age >= 18 and revenu > 2000:
    print('Crédit accordé')
else:
    print('Crédit refusé')
```

Sortie :

```
Note : 14/20 - Mention : Bien
Crédit accordé
```

```
# Condition en une ligne (ternaire)
age = 20
statut = 'majeur' if age >= 18 else 'mineur'
print(statut)
```

```
# Tester si une valeur est dans une liste
langages = ['Python', 'R', 'SQL', 'Scala']
print('Python' in langages)
print('Java' in langages)
```

```
# Tester si une valeur est None
valeur = None
if valeur is None:
    print('Valeur manquante')
```

Sortie :
majeur
True
False
Valeur manquante

1.4 Boucles

Les boucles permettent de répéter des actions. En Data Science, on parcourt souvent des listes, des colonnes, des fichiers ou des résultats de modèles.

```
# Boucle for : quand on sait combien de fois on répète
for i in range(5):          # i vaut 0, 1, 2, 3, 4
    print(i, end=' ')      # end=' ' pour ne pas aller à la ligne
print()                   # saut de ligne final
```

```
# Parcourir une liste
fruits = ['pomme', 'banane', 'cerise']
for fruit in fruits:
    print(f'Fruit : {fruit}')
```

Sortie :
0 1 2 3 4
Fruit : pomme
Fruit : banane
Fruit : cerise

```
# enumerate : index + valeur en même temps
for i, fruit in enumerate(['pomme', 'banane', 'cerise']):
    print(f'Index {i} : {fruit}')
```

```
# zip : parcourir deux listes simultanément
noms = ['Alice', 'Bob', 'Charlie']
scores = [85, 92, 78]
for nom, score in zip(noms, scores):
    print(f'{nom} : {score}/100')
```

Sortie :
Index 0 : pomme
Index 1 : banane
Index 2 : cerise
Alice : 85/100
Bob : 92/100
Charlie : 78/100

```
# Boucle while : tant qu'une condition est vraie
compteur = 0
while compteur < 4:
    print(f'Compteur : {compteur}')
    compteur += 1      # IMPORTANT : toujours faire évoluer la condition
```

```
# break : sortir de la boucle immédiatement
for i in range(10):
    if i == 5:
        break          # on sort quand i vaut 5
    print(i, end=' ')
```

```
print()

# continue : passer à l'itération suivante
for i in range(6):
    if i % 2 == 0:      # si pair, on saute
        continue
    print(i, end=' ') # on n'affiche que les impairs
```

Sortie :

```
Compteur : 0
Compteur : 1
Compteur : 2
Compteur : 3
0 1 2 3 4
1 3 5
```

1.5 Compréhensions de liste

La compréhension de liste est une syntaxe Python très élégante pour créer des listes en une seule ligne. Elle est plus rapide et plus lisible qu'une boucle for classique. C'est une écriture que tout Data Scientist utilise quotidiennement.

```
# Syntaxe : [expression for element in iterable if condition]

# Sans compréhension (méthode longue)
carres = []
for x in range(6):
    carres.append(x ** 2)
print(carres)

# Avec compréhension (méthode courte - préférée)
carres = [x ** 2 for x in range(6)]
print(carres)
```

Sortie :

```
[0, 1, 4, 9, 16, 25]
[0, 1, 4, 9, 16, 25]
```

```
# Avec condition (filtrer les éléments)
pairs = [x for x in range(11) if x % 2 == 0]
print(pairs)

# Transformer des chaînes
mots = [' data ', ' science ', ' python ']
mots_propres = [mot.strip().upper() for mot in mots]
print(mots_propres)

# Compréhension de dictionnaire
noms = ['alice', 'bob', 'charlie']
longueurs = {nom: len(nom) for nom in noms}
print(longueurs)

# Aplatir une liste de listes
matrice = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
plat = [val for ligne in matrice for val in ligne]
print(plat)
```

Sortie :

```
[0, 2, 4, 6, 8, 10]
['DATA', 'SCIENCE', 'PYTHON']
{'alice': 5, 'bob': 3, 'charlie': 7}
[1, 2, 3, 4, 5, 6, 7, 8, 9]
```

1.6 Classes et Programmation Orientée Objet

Une classe est un modèle pour créer des objets. Elle regroupe des données (attributs) et des actions (méthodes). En Data Science, vous rencontrerez des classes partout : les modèles sklearn, les DataFrames Pandas, etc. Savoir en créer vous permet de structurer votre propre code.

```
# Déclaration d'une classe
class Employe:
    # __init__ = constructeur, appelé à la création de l'objet
    def __init__(self, nom, salaire):
        self.nom = nom          # attribut d'instance
        self.salaire = salaire  # attribut d'instance

    # Méthode : une fonction définie dans la classe
    def augmenter(self, pourcentage):
        self.salaire *= (1 + pourcentage / 100)
        return self.salaire

    # __repr__ : comment l'objet s'affiche avec print()
    def __repr__(self):
        return f'Employe({self.nom}, {self.salaire}€)'

# Créer des instances (objets)
e1 = Employe('Alice', 3000)  # PAS new Employe() - pas de 'new' en Python
e2 = Employe('Bob', 2500)

print(e1)
print(e1.nom)
print(e1.augmenter(10))
print(e1)
```

Sortie :

```
Employe(Alice, 3000€)
Alice
3300.0
Employe(Alice, 3300.0€)
```

```
# Héritage : une classe enfant hérite d'une classe parent
class Manager(Employe):
    def __init__(self, nom, salaire, equipe):
        super().__init__(nom, salaire)  # appel du parent
        self.equipe = equipe

    def __repr__(self):
        return f'Manager({self.nom}, équipe={self.equipe})'

m = Manager('Claire', 5000, ['Alice', 'Bob'])
print(m)
print(m.augmenter(15))  # méthode héritée d'Employe
```

Sortie :

```
Manager(Claire, équipe=['Alice', 'Bob'])
5750.0
```

1.7 Gestion des fichiers

Lire des fichiers est une tâche très fréquente en Data Science : données CSV, fichiers texte, logs. Le mot-clé 'with' garantit que le fichier est toujours fermé proprement, même en cas d'erreur.

```
# Lire un fichier texte entier
with open('donnees.txt', 'r', encoding='utf-8') as f:
    contenu = f.read()  # tout le fichier en une chaîne
    print(contenu)

# Lire ligne par ligne (économise la mémoire)
with open('donnees.txt', 'r', encoding='utf-8') as f:
    for ligne in f:
        ligne = ligne.strip()  # supprimer \n en fin de ligne
```

```

        print(ligne)

# Lire un CSV simple sans pandas
with open('trainingdata.txt', 'r') as f:
    donnees = []
    for ligne in f:
        valeurs = ligne.strip().split(',')
        x = float(valeurs[0])
        y = float(valeurs[1])
        donnees.append((x, y))
print(f'{len(donnees)} lignes lues')

# Écrire dans un fichier
with open('resultats.txt', 'w', encoding='utf-8') as f:
    f.write('Résultats de l\'analyse\n')
    f.write(f'Nombre de lignes : {len(donnees)}\n')

```

Sortie :
100 lignes lues

1.8 Gestion des erreurs (try / except)

Un programme robuste ne plante pas sur une erreur inattendue. En Data Science, les données sont souvent imparfaites : valeurs manquantes, mauvais types, fichiers introuvables. Le try/except permet de gérer ces cas proprement.

```

# Structure de base
try:
    resultat = 10 / 0          # provoque une ZeroDivisionError
except ZeroDivisionError:
    print('Division par zéro impossible')
    resultat = 0

# Capturer plusieurs types d'erreurs
def convertir(valeur):
    try:
        return float(valeur)
    except ValueError:
        print(f'Impossible de convertir "{valeur}" en nombre')
        return None
    except TypeError:
        print(f'Type incorrect : {type(valeur)}')
        return None

print(convertir('3.14'))
print(convertir('abc'))
print(convertir(None))

```

Sortie :
Division par zéro impossible
3.14
Impossible de convertir "abc" en nombre
None
Type incorrect : <class 'NoneType'>
None

1.9 Modules et imports

Python est livré avec une grande bibliothèque standard. On importe des modules pour accéder à des fonctionnalités supplémentaires.

```

# Importer un module entier
import math
print(math.sqrt(16))      # racine carrée
print(math.pi)           # 3.14159...

```



```

print(math.ceil(2.3))    # arrondi supérieur
print(math.floor(2.9))  # arrondi inférieur

# Importer uniquement ce qu'on utilise
from math import sqrt, gcd, factorial
print(sqrt(25))
print(gcd(48, 18))      # Plus Grand Commun Diviseur
print(factorial(5))     # 5! = 5x4x3x2x1

# Importer avec un alias
import numpy as np      # convention universelle
import pandas as pd     # convention universelle

# Lister les fichiers d'un dossier
import os
fichiers = os.listdir('.') # dossier courant
print(fichiers)

```

Sortie :

```

4.0
3.141592653589793
3
2
5.0
6
120

```

1.10 if `__name__ == '__main__'` - Ordre d'exécution

Ce bloc est présent dans tout script Python professionnel. Il est important de comprendre son fonctionnement.

```

# Exemple complet

# Bloc A : s'exécute toujours (définit la fonction)
def traiter_donnees():
    print('Traitement en cours...') # Bloc B
    return 42

# Bloc C : s'exécute seulement si ce fichier est lancé directement
# PAS si ce fichier est importé par un autre script
if __name__ == '__main__':
    print('Démarrage du script')
    resultat = traiter_donnees()
    print(f'Résultat : {resultat}')

```

Sortie :

```

Démarrage du script
Traitement en cours...
Résultat : 42

```

Pourquoi c'est important

Quand on lance 'python mon_script.py' directement :

- `__name__` vaut '`__main__`' -> le bloc if s'exécute

Quand on importe le script dans un autre fichier :

- `__name__` vaut '`mon_script`' -> le bloc if ne s'exécute PAS

Cela permet de réutiliser les fonctions d'un script sans déclencher son code principal.

Ordre d'exécution avec 'python3 fichier.py' : A puis B (C déclenche l'appel de B).

PARTIE 2 - NumPy & Pandas : manipulation de données

NumPy et Pandas sont les deux bibliothèques les plus utilisées en Data Science. NumPy gère les calculs numériques rapides, Pandas gère les données structurées (tableaux). Un Data Scientist les utilise tous les jours.

2.1 NumPy - Calculs numériques vectorisés

NumPy introduit le concept d'array (tableau de nombres). La grande différence avec une liste Python : NumPy applique les opérations sur tous les éléments en une seule instruction, sans boucle. C'est beaucoup plus rapide.

```
import numpy as np

# Créer des arrays
a = np.array([1, 2, 3, 4, 5])
b = np.zeros(4)           # tableau de zéros
c = np.ones((2, 3))       # matrice 2x3 de 1
d = np.arange(0, 10, 2)   # de 0 à 9 avec pas de 2
e = np.linspace(0, 1, 5)  # 5 valeurs entre 0 et 1

print(a)
print(b)
print(c)
print(d)
print(e)
```

Sortie :

```
[1 2 3 4 5]
[0. 0. 0. 0.]
[[1. 1. 1.] [1. 1. 1.]]
[0 2 4 6 8]
[0.  0.25 0.5  0.75 1.  ]
```

```
# Opérations vectorisées : s'appliquent à tous les éléments
valeurs = np.array([3, 1, 4, 1, 5, 9, 2, 6])

print(valeurs * 2)           # multiplier chaque élément par 2
print(valeurs + 10)         # ajouter 10 à chaque élément
print(valeurs ** 2)         # carré de chaque élément
print(valeurs > 4)           # masque booléen
print(valeurs[valeurs > 4])  # filtrer les valeurs > 4
```

Sortie :

```
[ 6  2  8  2 10 18  4 12]
[13 11 14 11 15 19 12 16]
[ 9  1 16  1 25 81  4 36]
[False False False False  True  True False  True]
[5 9 6]
```

```
# Statistiques de base
data = np.array([3, 1, 4, 1, 5, 9, 2, 6, 5, 3])

print('Moyenne  :', np.mean(data))
print('Médiane  :', np.median(data))
print('Écart-type:', round(np.std(data), 2))
print('Variance  :', round(np.var(data), 2))
print('Min       :', np.min(data))
print('Max       :', np.max(data))
print('Somme    :', np.sum(data))

# reshape : changer la forme d'un array
ligne = np.array([1, 2, 3, 4, 5, 6])
matrice = ligne.reshape(2, 3) # 2 lignes, 3 colonnes
print(matrice)
```

```
# reshape(-1, 1) : transformer en colonne (utile pour sklearn)
colonne = ligne.reshape(-1, 1)
print(colonne.shape)
```

Sortie :

```
Moyenne : 3.9
Médiane : 3.5
Écart-type: 2.28
Variance : 5.29
Min : 1
Max : 9
Somme : 39
[[1 2 3] [4 5 6]]
(6, 1)
```

2.2 Pandas - DataFrames et manipulation de données

Un DataFrame Pandas est comme un tableau Excel intelligent. Chaque colonne est une Series (vecteur de données). C'est l'outil central de tout projet Data Science pour explorer, nettoyer et préparer les données.

```
import pandas as pd

# Créer un DataFrame manuellement
data = {
    'nom' : ['Alice', 'Bob', 'Claire', 'David'],
    'age' : [28, 34, 25, 41],
    'salaire': [3200, 4500, 2800, 5100],
    'ville' : ['Paris', 'Lyon', 'Paris', 'Bordeaux']
}
df = pd.DataFrame(data)
print(df)
```

Sortie :

	nom	age	salaire	ville
0	Alice	28	3200	Paris
1	Bob	34	4500	Lyon
2	Claire	25	2800	Paris
3	David	41	5100	Bordeaux

```
# Charger depuis un fichier CSV
df = pd.read_csv('fichier.csv')
df = pd.read_csv('fichier.csv', sep=';') # séparateur point-virgule
df = pd.read_csv('fichier.csv', encoding='utf-8') # encodage

# Explorer le DataFrame
print(df.shape) # (nombre_lignes, nombre_colonnes)
print(df.head(3)) # 3 premières lignes
print(df.tail(2)) # 2 dernières lignes
print(df.info()) # types de colonnes et valeurs manquantes
print(df.describe()) # statistiques : mean, std, min, max...
print(df.columns) # liste des noms de colonnes
print(df.dtypes) # type de chaque colonne
```

Sortie :

```
(4, 4)
   nom  age  salaire  ville
0  Alice   28    3200   Paris
...
```

```
# Accéder aux données
print(df['salaire']) # une colonne (Series)
print(df[['nom', 'salaire']]) # plusieurs colonnes

# Filtrer les lignes
parisiens = df[df['ville'] == 'Paris']
```

```
print(parisiens)

seniors = df[df['age'] > 30]
print(seniors)

# Filtre combiné
bien_payes = df[(df['salaire'] > 4000) & (df['age'] < 40)]
print(bien_payes)
```

Sortie :

```
   nom  age  salaire  ville
0  Alice  28    3200  Paris
2  Claire  25    2800  Paris
   nom  age  salaire  ville
1  Bob   34    4500   Lyon
3 David  41    5100 Bordeaux
   nom  age  salaire  ville
1  Bob   34    4500   Lyon
```

```
# Créer et modifier des colonnes
df['salaire_annuel'] = df['salaire'] * 12
df['senior'] = df['age'] > 35
df['categorie'] = df['salaire'].apply(lambda x: 'élevé' if x > 4000 else 'moyen')
print(df[['nom', 'salaire', 'salaire_annuel', 'categorie']])

# Trier
df_trie = df.sort_values('salaire', ascending=False)
print(df_trie[['nom', 'salaire']])
```

Sortie :

```
   nom  salaire  salaire_annuel  categorie
0  Alice    3200         38400      moyen
1   Bob    4500         54000      élevé
2  Claire    2800         33600      moyen
3  David    5100         61200      élevé
   nom  salaire
3 David    5100
1   Bob    4500
0 Alice    3200
2 Claire    2800
```

```
# Gestion des valeurs manquantes
print(df.isnull().sum())      # compter les NaN par colonne
print(df.isnull().sum().sum()) # total NaN dans tout le DataFrame

df_propre = df.dropna()      # supprimer les lignes avec NaN
df['age'] = df['age'].fillna(df['age'].median()) # remplir par médiane
df['ville'] = df['ville'].fillna('Inconnue')    # remplir par valeur fixe

# Supprimer les doublons
df = df.drop_duplicates()

# Grouper et agréger
print(df.groupby('ville')['salaire'].mean())
print(df.groupby('ville').agg({
    'salaire': ['mean', 'max'],
    'age'     : 'mean'
})))
```

Sortie :

```
Bordeaux    5100.0
Lyon         4500.0
Paris        3000.0
Name: salaire, dtype: float64
```

2.3 La fonction average() - Exercice fondamental

Calculer la moyenne est une opération basique mais il faut toujours gérer le cas de la liste vide pour éviter une division par zéro.

```
# Version Python pure
def average(table):
    if len(table) == 0:      # toujours gérer le cas vide
        return 0
    return sum(table) / len(table)

print(average([3, 1, 3, 4]))
print(average([]))

# Version NumPy (équivalente)
import numpy as np

def average_numpy(table):
    if len(table) == 0:
        return 0
    return np.mean(table)

print(average_numpy([10, 20, 30]))
print(average_numpy([]))
```

Sortie :

```
2.75
0
20.0
0
```

PARTIE 3 - Scikit-learn : Machine Learning en pratique

Scikit-learn (sklearn) est la bibliothèque ML de référence en Python. Elle propose des algorithmes prêts à l'emploi avec une interface unifiée : `fit()` pour entraîner, `predict()` pour prédire, `score()` pour évaluer.

3.1 Pipeline complet d'un projet ML

Voici le squelette que tout Data Scientist reproduit pour chaque projet. L'ordre des étapes est important et ne doit pas être modifié.

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, root_mean_squared_error, r2_score

# Étape 1 : Charger les données
df = pd.read_csv('donnees.csv')

# Étape 2 : Séparer features (X) et target (y)
X = df.drop('prix', axis=1) # tout sauf la colonne cible
y = df['prix']             # la colonne à prédire

# Étape 3 : Diviser en Train et Test (AVANT de normaliser !)
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,      # 20% pour le test
    random_state=42     # pour la reproductibilité
)
print(f'Train : {X_train.shape[0]} lignes')
print(f'Test  : {X_test.shape[0]} lignes')
```

Sortie :

```
Train : 800 lignes
Test  : 200 lignes
```

```
# Étape 4 : Normaliser (fit UNIQUEMENT sur Train, transform sur les deux)
# ATTENTION : ne jamais faire fit_transform sur X_test !
# Sinon on introduit du Data Leakage
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train) # apprend + transforme
X_test  = scaler.transform(X_test)      # transforme seulement

# Étape 5 : Choisir et entraîner le modèle
model = LinearRegression()
model.fit(X_train, y_train)

# Étape 6 : Prédire
y_pred = model.predict(X_test)

# Étape 7 : Évaluer
rmse = root_mean_squared_error(y_test, y_pred) # sklearn >= 1.4
r2    = r2_score(y_test, y_pred)
print(f'RMSE : {rmse:.2f}')
print(f'R²   : {r2:.4f}')
```

Sortie :

```
RMSE : 45230.12
R²   : 0.7842
```

La règle d'or du preprocessing

Toujours dans cet ordre :

1. Split Train/Test
2. `scaler.fit_transform(X_train)` - le scaler apprend sur Train
3. `scaler.transform(X_test)` - applique les mêmes paramètres

Si on fait `fit_transform` sur `X_test`, le scaler 'voit' les données de test pendant l'apprentissage -> Data Leakage -> performances surestimées.

3.2 Les principaux algorithmes sklearn

```
from sklearn.linear_model import LinearRegression, LogisticRegression, Ridge, Lasso
from sklearn.ensemble import RandomForestClassifier, RandomForestRegressor
from sklearn.ensemble import GradientBoostingClassifier, GradientBoostingRegressor
from sklearn.svm import SVC, SVR
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.cluster import KMeans, DBSCAN
from sklearn.decomposition import PCA

# Tous les modèles supervisés ont la même interface :
# model.fit(X_train, y_train) -> entraîner
# model.predict(X_test) -> prédire
# model.score(X_test, y_test) -> évaluer (R² ou accuracy)

# Exemple Random Forest (recommandé comme baseline)
rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X_train, y_train)
print(f'Accuracy : {rf.score(X_test, y_test):.4f}')
```

Accéder à l'importance des features

```
importances = pd.Series(rf.feature_importances_, index=df.columns[:-1])
print(importances.sort_values(ascending=False).head(5))
```

Sortie :

```
Accuracy : 0.8750
revenu    0.3421
age       0.2187
anciennete 0.1543
...
```

3.3 Métriques d'évaluation complètes

```
from sklearn.metrics import (
    # Régression
    mean_squared_error,
    root_mean_squared_error,    # sklearn >= 1.4
    mean_absolute_error,
    r2_score,
    # Classification
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report,
    roc_auc_score
)

# --- RÉGRESSION ---
mse = mean_squared_error(y_test, y_pred)
rmse = root_mean_squared_error(y_test, y_pred)    # sklearn >= 1.4
mae = mean_absolute_error(y_test, y_pred)
```

```

r2 = r2_score(y_test, y_pred)
print(f'MSE : {mse:.2f}')
print(f'RMSE : {rmse:.2f}')
print(f'MAE : {mae:.2f}')
print(f'R² : {r2:.4f}')

```

Sortie :

```

MSE : 2045793024.00
RMSE : 45230.12
MAE : 32450.87
R² : 0.7842

```

```

# --- CLASSIFICATION ---
acc = accuracy_score(y_test, y_pred_class)
prec = precision_score(y_test, y_pred_class)
rec = recall_score(y_test, y_pred_class)
f1 = f1_score(y_test, y_pred_class)
auc = roc_auc_score(y_test, y_pred_proba)

print(f'Accuracy : {acc:.4f}')
print(f'Precision : {prec:.4f}')
print(f'Recall : {rec:.4f}')
print(f'F1-Score : {f1:.4f}')
print(f'AUC-ROC : {auc:.4f}')

# Rapport complet (très utile pour présenter les résultats)
print(classification_report(y_test, y_pred_class))

# Matrice de confusion
cm = confusion_matrix(y_test, y_pred_class)
print(cm)

```

Sortie :

```

Accuracy : 0.8750
Precision : 0.8621
Recall : 0.8824
F1-Score : 0.8721
AUC-ROC : 0.9234

```

	precision	recall	f1-score	support
0	0.88	0.86	0.87	100
1	0.86	0.88	0.87	100

```

[[86 14]
 [12 88]]

```

3.4 Cross-validation

La cross-validation évalue un modèle de façon plus robuste qu'un simple split. Elle entraîne et teste k fois sur des parties différentes des données.

```

from sklearn.model_selection import cross_val_score, KFold, StratifiedKFold
from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier(n_estimators=100, random_state=42)

# K-Fold standard
kf = KFold(n_splits=5, shuffle=True, random_state=42)
scores = cross_val_score(model, X, y, cv=kf, scoring='accuracy')

print('Scores par fold :', scores.round(3))
print(f'Moyenne : {scores.mean():.4f}')
print(f'Écart-type : {scores.std():.4f}')

# StratifiedKFold : conserve les proportions de classes
# Recommandé pour les problèmes de classification déséquilibrés
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
scores_s = cross_val_score(model, X, y, cv=skf, scoring='f1')

```



```
print(f'F1 moyen : {scores_s.mean():.4f}')
```

```
# ATTENTION : sur les séries temporelles, utiliser TimeSeriesSplit
# pour respecter l'ordre chronologique (pas de mélange passé/futur)
from sklearn.model_selection import TimeSeriesSplit
tscv = TimeSeriesSplit(n_splits=5)
```

Sortie :

```
Scores par fold : [0.872 0.885 0.863 0.891 0.878]
Moyenne : 0.8778
Écart-type : 0.0101
F1 moyen : 0.8712
```

3.5 Régression linéaire sur un fichier texte

Cas pratique très fréquent : lire un fichier de données, entraîner un modèle simple, créer une fonction de prédiction.

```
import numpy as np
from sklearn.linear_model import LinearRegression

# Lire le fichier (format : valeur1,valeur2 par ligne)
X_data, y_data = [], []
with open('trainingdata.txt', 'r') as f:
    for ligne in f:
        valeurs = ligne.strip().split(',')
        X_data.append(float(valeurs[0]))
        y_data.append(float(valeurs[1]))

# Préparer les arrays NumPy
X_train = np.array(X_data).reshape(-1, 1) # (n, 1) requis par sklearn
y_train = np.array(y_data)

# Entraîner le modèle UNE SEULE FOIS (en dehors de la fonction)
model = LinearRegression()
model.fit(X_train, y_train)
print(f'Coefficient : {model.coef_[0]:.4f}')
print(f'Intercept : {model.intercept_:.4f}')
print(f'R² : {model.score(X_train, y_train):.4f}')

# Fonction de prédiction
def prediction(valeur_entree):
    X_new = np.array([[valeur_entree]])
    pred = model.predict(X_new)[0]
    return round(pred) # arrondi à l'entier

print(prediction(4.7))
print(prediction(2.0))
```

Sortie :

```
Coefficient : 1.7842
Intercept : -0.3421
R² : 0.9234
8
3
```

PARTIE 4 - Exercices pratiques avec solutions

Ces exercices couvrent les situations que rencontre un Data Scientist au quotidien. Pour chaque exercice : lisez l'énoncé, essayez de résoudre, puis consultez la solution.

Exercice 1 - Calculer une moyenne robuste

Énoncé	Créez une fonction <code>average(table)</code> qui retourne la moyenne d'une liste de nombres. Elle doit retourner 0 si la liste est vide (pas de division par zéro).
Solution	<pre>def average(table): # Cas limite : liste vide if len(table) == 0: return 0 # Calcul de la moyenne return sum(table) / len(table) print(average([3, 1, 3, 4])) print(average([])) print(average([10]))</pre> Sortie : 2.75 0 10.0
Explication	<i>Toujours gérer le cas vide AVANT de calculer. <code>sum()</code> additionne tous les éléments, <code>len()</code> compte les éléments. Sans le <code>if</code>, <code>average([])</code> provoquerait une <code>ZeroDivisionError</code> et ferait planter le programme.</i>

Exercice 2 - Compter les paires minuscules

Énoncé	On parcourt deux tableaux <code>a</code> (gauche à droite) et <code>b</code> (droite à gauche) simultanément. Une paire <code>(x, y)</code> est 'minuscule' si la concaténation <code>str(x)+str(y)</code> convertie en entier est $< k$. Compter ces paires.
Solution	<pre>def countTinyPairs(a, b, k): n = len(a) n_tinypairs = 0 for i in range(n): x = a[i] y = b[n - 1 - i] # b parcouru à l'envers # Concaténer comme chaînes puis comparer concat = int(str(x) + str(y)) if concat < k: n_tinypairs += 1 return n_tinypairs print(countTinyPairs([1,2,3],[1,2,3], 31)) print(countTinyPairs([16,1,4,2,14],[7,11,2,0,15], 743))</pre> Sortie : 2 4
Explication	<i>L'astuce clé : la concaténation se fait sur les CHAÎNES et non par addition numérique. <code>int(str(16)+str(15)) = int('1615') = 1615</code>, PAS <code>16+15=31</code>. Pour parcourir <code>b</code> à l'envers, l'index est <code>n-1-i</code>.</i>

Exercice 3 - Calculer une probabilité exacte

Énoncé	Lors d'un lancer de deux dés à 6 faces équilibrés, quelle est la probabilité que leur somme soit au plus égale à 9 ? Répondre sous forme de fraction irréductible.
Solution	<pre>from math import gcd # Énumérer tous les cas possibles favorable = 0</pre>

	<pre> total = 0 for d1 in range(1, 7): # dé 1 : valeurs 1 à 6 for d2 in range(1, 7): # dé 2 : valeurs 1 à 6 total += 1 if d1 + d2 <= 9: # condition : somme <= 9 favorable += 1 # Simplifier la fraction diviseur = gcd(favorable, total) num = favorable // diviseur den = total // diviseur print(f'{favorable}/{total} = {num}/{den}') Sortie : 30/36 = 5/6 </pre>
Explication	On énumère toutes les 36 combinaisons (6x6). Les cas où somme > 9 sont : (4+6), (5+5), (5+6), (6+4), (6+5), (6+6) = 6 cas. Donc 30 cas favorables. gcd() du module math calcule le Plus Grand Commun Diviseur pour simplifier la fraction.

Exercice 4 - Nettoyage et analyse d'un DataFrame

Énoncé	À partir d'un DataFrame avec des valeurs manquantes, imputer les valeurs numériques par la médiane, les catégorielles par le mode, puis afficher les statistiques par groupe.
Solution	<pre> import pandas as pd import numpy as np # Données avec valeurs manquantes data = { 'age' : [25, np.nan, 35, 28, np.nan, 42], 'salaire' : [3000, 4500, np.nan, 3200, 5000, 4800], 'ville' : ['Paris', 'Lyon', None, 'Paris', 'Lyon', None] } df = pd.DataFrame(data) # Imputation numérique : médiane df['age'] = df['age'].fillna(df['age'].median()) df['salaire'] = df['salaire'].fillna(df['salaire'].median()) # Imputation catégorielle : mode (valeur la plus fréquente) df['ville'] = df['ville'].fillna(df['ville'].mode()[0]) print(df) print(df.groupby('ville')['salaire'].mean().round(0)) Sortie : age salaire ville 0 25.0 3000.0 Paris 1 31.5 4500.0 Lyon 2 35.0 4650.0 Paris 3 28.0 3200.0 Paris 4 31.5 5000.0 Lyon 5 42.0 4800.0 Paris ville Lyon 4750.0 Paris 3912.0 </pre>
Explication	La médiane est préférable à la moyenne pour imputer car elle est robuste aux outliers. Le mode (valeur la plus fréquente) est le choix standard pour les variables catégorielles. fillna() remplace toutes les valeurs NaN en une seule instruction.

Exercice 5 - Pipeline sklearn complet

Énoncé	Créer un Pipeline sklearn qui enchaîne preprocessing et modèle, effectue une cross-validation et affiche les performances.
Solution	<pre> from sklearn.pipeline import Pipeline from sklearn.preprocessing import StandardScaler from sklearn.ensemble import RandomForestClassifier from sklearn.model_selection import cross_val_score import numpy as np </pre>

```
# Le Pipeline garantit que le scaler ne voit jamais le test
pipe = Pipeline([
    ('scaler', StandardScaler()),
    ('model', RandomForestClassifier(n_estimators=100, random_state=42))
])

# Cross-validation directement sur le Pipeline
scores = cross_val_score(pipe, X, y, cv=5, scoring='accuracy')
print('Scores :', scores.round(3))
print(f'Moyenne    : {scores.mean():.4f}')
print(f'Écart-type: {scores.std():.4f}')

# Entraîner sur toutes les données et prédire
pipe.fit(X_train, y_train)
y_pred = pipe.predict(X_test)
Sortie :
Scores : [0.875 0.888 0.862 0.891 0.879]
Moyenne    : 0.8790
Écart-type: 0.0102
```

Explication

Le Pipeline est la bonne pratique absolue : il empêche le Data Leakage en s'assurant que le scaler est entraîné uniquement sur les données d'entraînement de chaque fold. C'est aussi plus lisible et maintenable.

PARTIE 5 - Questions théoriques essentielles

Ces questions testent la compréhension conceptuelle du Machine Learning. Un Data Scientist doit savoir y répondre clairement et justifier sa réponse.

5.1 Tableau complet des réponses et justifications

Déclarer une fonction en Python	def nom(): - PAS function nom() ni void nom()
Type de {'0':0, '1':1}	dict - accolades + clé:valeur = dictionnaire
Créer une instance Point(10,20)	point = Point(10, 20) - PAS new Point() comme en Java
Opérateur ET en Python	and - PAS && qui est la syntaxe C/JavaScript
Tuples vs Lists	Tuples = immuables (non modifiables). Lists = mutables (modifiables).
Ordre d'exécution if __main__	A puis B. A = définition, le bloc if déclenche l'appel de la fonction.
max([0,1,2]) - quelle fonction	max(), np.max() et pd.Series().max() fonctionnent tous les trois.
Dossier vide - que retourner	Retourner [] (liste vide). PAS None, PAS lever une exception.
K-Means - ce qui est nécessaire	D - Tout : centroïdes initiaux + nombre k + métrique de distance.
Supervisé vs Non supervisé	Supervisé nécessite des données étiquetées. Non supervisé non.
Compromis Biais/Variance	Biais = sous-apprentissage. Variance = surapprentissage. Dilemme fondamental.
Matrice de confusion - à quoi sert	Évaluer les performances d'un modèle de classification.
Recall - formule exacte	TP / (TP + FN) - parmi tous les vrais positifs, combien détectés ?
Cross-validation + séries temporelles	FAUX - utiliser TimeSeriesSplit pour respecter l'ordre chronologique.
Lutter contre l'overfitting	L1/L2 + Cross-validation + PCA. Ajouter des arbres n'aide pas.
Bagging vs Boosting	Bagging = parallèle (Random Forest). Boosting = séquentiel (XGBoost).

5.2 Les concepts à expliquer avec ses propres mots

K-Means - Fonctionnement et pré-requis

K-Means partitionne les données en k groupes en minimisant la distance de chaque point à son centroïde. Trois éléments sont nécessaires :

1. Le nombre k de groupes (à spécifier avant l'entraînement)
2. Une initialisation des centroïdes (aléatoire ou K-Means++)
3. Une métrique de distance (euclidienne par défaut)

L'algorithme converge quand les centroïdes ne bougent plus.

Biais / Variance - Le dilemme fondamental

Tout modèle ML souffre de ce compromis impossible :

- Modèle trop simple (haut biais) : sous-apprend, rate les patterns = underfitting
- Modèle trop complexe (haute variance) : mémorise les données = overfitting

L'objectif est de trouver le meilleur compromis, pas d'éliminer l'un ou l'autre.

Erreur totale = Biais² + Variance + Bruit irréductible

Recall vs Precision - Quand utiliser lequel ?

Recall = $TP / (TP + FN)$: sur tous les vrais positifs, combien ai-je trouvés ?

- À maximiser quand rater un positif est grave (cancer, fraude, incendie)
- On préfère avoir des fausses alarmes plutôt que de rater un vrai cas

Precision = $TP / (TP + FP)$: sur mes alertes, combien sont vraiment positives ?

- À maximiser quand une fausse alarme est coûteuse (spam, fausse accusation)
- On préfère être certain avant d'alerter

F1-Score = équilibre entre les deux. Recommandé sur les datasets déséquilibrés.

Bagging vs Boosting - La différence clé

Bagging (Bootstrap Aggregating) - exemple : Random Forest

- Entraîne plusieurs modèles EN PARALLÈLE sur des sous-échantillons aléatoires
- Vote final à la majorité (classification) ou moyenne (régression)
- Réduit la variance, robuste aux outliers

Boosting - exemples : XGBoost, LightGBM, AdaBoost

- Entraîne les modèles EN SÉRIE, chaque modèle corrige les erreurs du précédent
- Généralement plus performant mais plus sensible à l'overfitting
- Nécessite un réglage plus fin des hyperparamètres

PARTIE 6 - Aide-mémoire express

6.1 Syntaxe Python - Les erreurs à ne pas faire

Déclarer une fonction	def ma_fonction(x): - PAS function, PAS void
Créer un objet	obj = MaClasse(args) - PAS new MaClasse()
ET logique	and - PAS &&
OU logique	or - PAS
NON logique	not - PAS !
Commentaire	# mon commentaire - PAS // ni /* */
Indentation	4 espaces obligatoires - PAS des accolades {}
F-string	f'Bonjour {nom}' - formatage moderne de chaînes
Exposant (puissance)	x**2 - PAS x^2 (^ = XOR en Python, pas la puissance)
Division entière	x // y - retourne le quotient sans reste
Modulo (reste)	x % y - retourne le reste de la division
Longueur	len(ma_liste) - fonctionne sur listes, dicts, strings

6.2 Fonctions Python natives incontournables

```
# Nombres et mathématiques
abs(-5)          # 5          - valeur absolue
round(2.758, 2)  # 2.76      - arrondir à 2 décimales
pow(2, 10)       # 1024      - puissance (= 2**10)
divmod(17, 5)    # (3, 2)    - quotient et reste

# Listes et itérables
len([1,2,3,4])   # 4         - longueur
sum([1,2,3,4])   # 10        - somme
max([1,2,3,4])   # 4         - maximum
min([1,2,3,4])   # 1         - minimum
sorted([3,1,4,1,5]) # [1,1,3,4,5] - trier (ne modifie pas l'original)
list(reversed([1,2,3])) # [3,2,1]
list(range(1, 6)) # [1,2,3,4,5]
list(range(0,10,2)) # [0,2,4,6,8] - de 0 à 9 par pas de 2

# Manipulation de chaînes
'hello world'.upper() # 'HELLO WORLD'
'HELLO'.lower()       # 'hello'
' hello '.strip()     # 'hello'
'a,b,c'.split(',')    # ['a','b','c']
','.join(['a','b','c']) # 'a,b,c'
'hello world'.replace('o','0') # 'hell0 w0rld'
'hello'.startswith('he') # True
f'Pi vaut environ {3.14159:.2f}' # 'Pi vaut environ 3.14'

# Conversions de types
int('42')          # 42        - str -> int
float('3.14')      # 3.14      - str -> float
str(42)            # '42'       - int -> str
list((1, 2, 3))    # [1,2,3]   - tuple -> list
tuple([1, 2, 3])   # (1,2,3)   - list -> tuple
set([1,2,2,3])     # {1,2,3}   - list -> set (dédoublonne)
```

```
# Utilitaires
type(42)          # <class 'int'>
isinstance(42, int) # True
print(repr('hello')) # 'hello' (avec les guillemets)
```

6.3 Imports essentiels à mémoriser

```
# Mathématiques
import math
math.sqrt(25)      # 5.0    - racine carrée
math.gcd(48, 18)   # 6      - Plus Grand Commun Diviseur
math.pi            # 3.14159...
math.e             # 2.71828... - nombre d'Euler
math.ceil(2.1)     # 3      - arrondi supérieur
math.floor(2.9)    # 2      - arrondi inférieur
math.log(100, 10)  # 2.0    - log base 10
math.exp(1)        # 2.71828... - exponentielle

# NumPy
import numpy as np
np.mean(arr)       # moyenne
np.std(arr)        # écart-type
np.median(arr)     # médiane
np.percentile(arr, 75) # 3ème quartile
np.corrcoef(x, y)   # matrice de corrélation
np.log(arr)        # logarithme naturel
np.exp(arr)        # exponentielle
np.random.seed(42) # graine aléatoire (reproductibilité)

# Pandas
import pandas as pd
pd.read_csv()      # lire un CSV
pd.DataFrame()     # créer un DataFrame
pd.concat()        # concaténer plusieurs DataFrames
pd.merge()         # jointure entre DataFrames
pd.get_dummies()   # One-Hot Encoding rapide

# Scikit-learn
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.preprocessing import StandardScaler, MinMaxScaler, LabelEncoder
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LinearRegression, LogisticRegression
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.metrics import r2_score, f1_score, confusion_matrix

# Visualisation
import matplotlib.pyplot as plt
import seaborn as sns
plt.figure(figsize=(10, 6))
sns.heatmap(df.corr(), annot=True) # carte de chaleur des corrélations
plt.show()
```

6.4 Les 5 erreurs les plus coûteuses

Erreur 1 : Oublier le return dans une fonction

```
def ma_fonction(x):
    x * 2    # pas de return -> retourne None
```

Correction : return x * 2

Erreur 2 : fit_transform sur le jeu de test (Data Leakage)

```
# FAUX - le scaler 'voit' les données de test
X_train = scaler.fit_transform(X_train)
X_test = scaler.fit_transform(X_test) # FAUX !

# CORRECT - le scaler n'apprend que sur Train
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test) # CORRECT
```

Erreur 3 : Division par zéro sur liste vide

```
def average(table):
    return sum(table) / len(table) # plante si table est vide !
```

Correction : if len(table) == 0: return 0

Erreur 4 : Utiliser && ou || au lieu de and / or

```
if x > 0 && x < 10: # SyntaxError en Python
if x > 0 || x < 10: # SyntaxError en Python
```

Correction : if x > 0 and x < 10:
if x > 0 or x < 10:

Erreur 5 : Ne pas fixer random_state

```
# Sans random_state : résultats différents à chaque exécution
X_train, X_test = train_test_split(X, y)

# Avec random_state : résultats toujours identiques
X_train, X_test = train_test_split(X, y, test_size=0.2, random_state=42)
model = RandomForestClassifier(n_estimators=100, random_state=42)
```

6.5 Checklist avant de livrer un projet

Données

- Explorer avec df.head(), df.info(), df.describe()
- Vérifier et traiter les valeurs manquantes df.isnull().sum()
- Vérifier et supprimer les doublons df.duplicated().sum()
- Analyser la distribution de la variable cible
- Encoder les variables catégorielles
- Normaliser/standardiser les variables numériques

Modélisation

- Splitter AVANT de normaliser (train_test_split en premier)
- Utiliser random_state=42 partout
- Commencer par un modèle simple (baseline) avant les modèles complexes
- Évaluer avec cross-validation (pas seulement sur le Test)
- Choisir la bonne métrique selon le problème (F1 si déséquilibre)

Code

- Toutes les fonctions ont un return
- Les cas limites sont gérés (liste vide, valeur None, division par zéro)
- Les imports sont en haut du fichier

- Les variables ont des noms clairs et explicites
- Le code est commenté aux endroits importants